SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

Course Name:	Software Testing	**Course Code:**	CSA37
Student Name:	Ganesh Karthick M	**Reg. No.:**	192321138
Section / Batch:		**Date:**	24-10-2026

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

QUESTIONS: 10

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5
4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	—
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5

Sl. No	Questions	Bloom's Level
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5
10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

I Ganesh Karthick M / 192321138 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Annexure A – Error Analysis Task: Answers

The questions describe faulty test designs without giving the data, so each answer states the faulty design it assumes, lists its errors and root causes, and then shows the corrected design.

Question 1: Equivalence Partitioning and Boundary Value Errors

Assumed rule: order quantity must be an integer from 1 to 10. **Faulty design:** Class A = 0 to 5, Class B = 6 to 15; one random value from each class (3 and 12); no boundary values.

1. Errors identified

No.	Error	Explanation
E1	Valid and invalid values mixed in one class	Class A contains 0 (invalid) and 1 to 5 (valid); Class B contains 6 to 10 (valid) and 11 to 15 (invalid)
E2	Classes based on convenient ranges, not on behaviour	Members of a class must all be treated the same way; here they are not
E3	Missing invalid classes	Negative, non-integer (2.5), non-numeric (abc) and empty values are not partitioned
E4	Values above 15 not covered	The partitioning does not span the whole input domain
E5	No boundary value analysis	0, 1, 10 and 11 are never tested

2. Root causes

- The tester partitioned the numbers without first writing the requirement as valid and invalid sets.
- The principle of equivalence partitioning (every member of a class behaves identically) was not applied or checked.
- Defects occur mostly at the edges of ranges, but boundary analysis was treated as optional.

3. Corrected partitions and test cases

Class	Range	Valid?	Representative
C1	< 1 (zero or negative)	No	−5
C2	1 to 10	Yes	5
C3	> 10	No	50
C4	Non-integer	No	2.5
C5	Non-numeric / empty	No	abc

TC	Quantity	Technique	Expected result
1	−5	ECP C1	Rejected
2	0	BVA min−	Rejected
3	1	BVA min	Accepted
4	2	BVA min+	Accepted
5	5	ECP C2	Accepted
6	9	BVA max−	Accepted
7	10	BVA max	Accepted
8	11	BVA max+	Rejected
9	50	ECP C3	Rejected

TC	Quantity	Technique	Expected result
10	2.5	ECP C4	Rejected
11	abc	ECP C5	Rejected

4. Improvement in defect detection

Seeded defect	Faulty tests (3, 12)	Corrected tests
Code uses quantity < 10 (rejects 10)	Missed	Found by TC 7
Code uses quantity ≤ 11 (accepts 11)	Missed (12 is rejected anyway)	Found by TC 8
Code uses quantity > 1 (rejects 1)	Missed	Found by TC 3
Detected	0 of 3	3 of 3

Interpretation: correct classification removes ambiguity and boundary values target the exact points where off-by-one defects exist, so detection rises from 0 of 3 to 3 of 3 seeded defects with only 11 tests.

Question 2: Random Testing in a Banking Application

Assumed rule: a transaction amount must be between 100 and 50,000. The QA engineer ran 1,000 random amounts drawn from 1 to 1,00,000 and reported "high coverage".

1. Errors in strategy and coverage assumptions

No.	Error	Why it is wrong
E1	Number of tests treated as coverage	Coverage is a measure against requirements, partitions or code, not a count of tests
E2	No technique, so no traceability	Tests cannot be linked to limits, balance or daily-limit rules
E3	Boundaries left to chance	Random values rarely land on the exact limits
E4	Not repeatable or reviewable	Next release may test entirely different values
E5	No expected results defined	Pass/fail cannot be judged without an oracle

2. Root cause analysis (quantified)

The boundary values are 99, 100, 101, 49,999, 50,000 and 50,001: 6 values out of 1,00,000 possible amounts. The probability that one random test hits any of them is 6 / 100,000 = 0.006%. For 1,000 random tests the probability of hitting at least one boundary is $1 - (1 - 0.00006)^{1000} \approx 1 - e^{-0.06} \approx$ **5.8%**, and hitting all six is practically zero. So the missed limit defects were expected, not bad luck.

3. Appropriate black box techniques

Technique	Application	Ensures
Equivalence partitioning	Below limit, within limit, above limit, non-numeric	Every behaviour class tested at least once
Boundary value analysis	99, 100, 101, 49,999, 50,000, 50,001	Off-by-one defects at limits found
Decision table	Amount in limit x balance sufficient x daily limit not exceeded (8 rules)	All condition combinations tested
State / sequence testing	Several transactions reaching the daily limit	Cumulative limits behave correctly

4. Justification

With these techniques, adequacy is **measurable**: 4 of 4 partitions and 6 of 6 boundaries covered, 8 of 8 decision rules covered, with about 20 tests instead of 1,000 random ones. The tests are repeatable and traceable to requirements, so the suite is reliable across releases. Random testing may still be kept as an extra exploratory step after the structured tests, not as a replacement. **Interpretation:** structured techniques turn an unmeasurable claim of "high coverage" into a verifiable one.

Question 3: Incomplete Control Flow Graph and Path Coverage

Assumed module (order processing): while there are orders, if the order is valid then (express or standard shipping, then a bonus if total > 1000) else log an error; i++. The developer drew a straight line: 1 → 3 → 4 → 5 → 7 → 8 → 10 → 11.

1. Errors in the developer's graph

No.	Error	Effect
E1	Loop decision node (i < n) missing	The loop and its exit were not represented; loop tests were never designed
E2	Back edge from the loop end to the loop test missing	Repeated iterations not modelled
E3	Decision nodes drawn as plain statements (valid?, express?, total > 1000?)	False outcomes ignored
E4	Else/alternative nodes missing (log error, standard shipping)	Those statements never targeted

The faulty graph has N = 8 nodes and E = 7 edges, so V(G) = 7 − 8 + 2 = **1** path. The module actually has 4 decisions, so the correct V(G) = 4 + 1 = **5**. A test following the one drawn path covers 9 of 11 statements (82%) and 9 of 14 edges (64%), and only 1 of 5 independent paths (20%).

2. Root causes

- Graph built by reading the happy path, not by identifying every decision and loop in the code.
- Misunderstanding of CFG rules: each decision must produce two outgoing edges, and each loop needs a test node and a back edge.
- No check of the graph against V(G) = decisions + 1 or E − N + 2.

3. Corrected CFG and independent paths

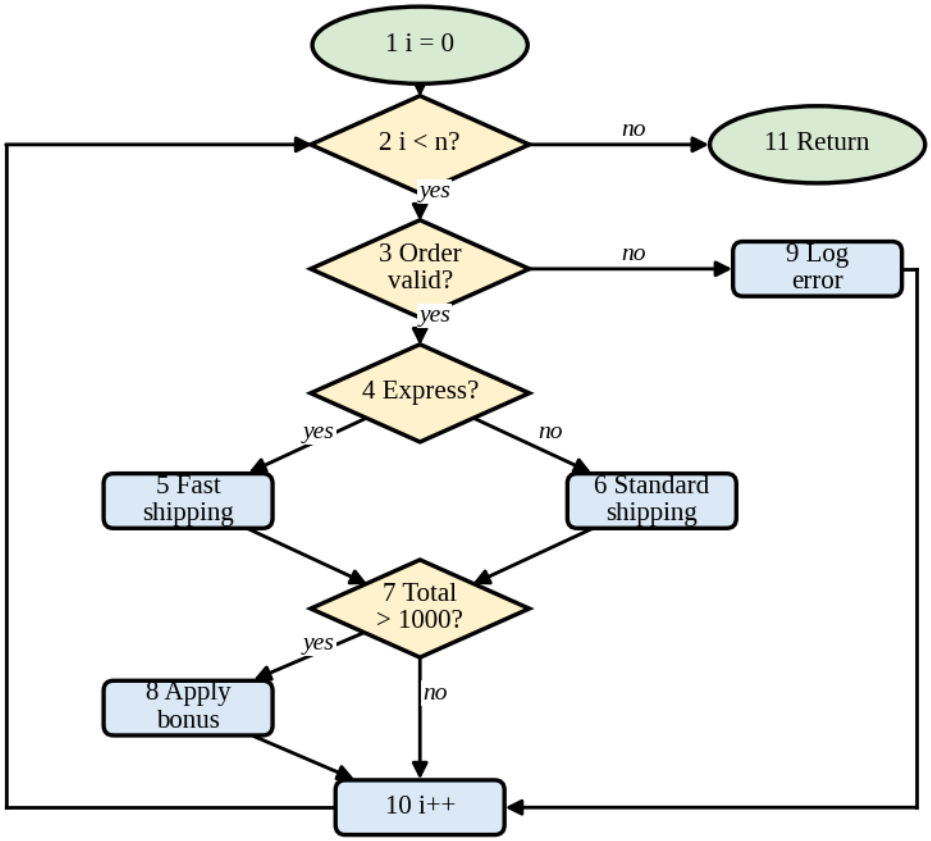


Figure 1: Corrected control flow graph of the order-processing module

E = 14, N = 11, V(G) = 14 − 11 + 2 = 5.

Path	Node sequence	Scenario	Test input
P1	1-2-11	No orders	n = 0
P2	1-2-3-9-10-2-11	Invalid order	1 invalid order
P3	1-2-3-4-5-7-10-2-11	Express, total ≤ 1000	express, 800
P4	1-2-3-4-6-7-10-2-11	Standard, total ≤ 1000	standard, 800
P5	1-2-3-4-5-7-8-10-2-11	Express, total > 1000	express, 1500

4. Justification

The five tests exercise all 14 edges (100% branch coverage) and all 11 statements; the loop is tested for zero and for one or more iterations; and the count V(G) = 5 gives an objective target, so a missing test is immediately visible. Defects on the error path, the standard shipping path and the bonus path, which the single-path test could never reach, are now detectable. **Interpretation:** correct path testing starts with a complete graph.

Question 4: 100% Statement Coverage Missing Logical Errors

Assumed module:

```
1: discount = 0
2: if (amount > 1000 || member) discount = 10
3: if (coupon && amount > 500) discount += 5
4: return discount
```

1. Adequacy of the testing performed

One test, **amount 2000, member false, coupon true**, executes every statement (discount 15), so the team reports 100% statement coverage. But both decisions are only ever true: the false outcomes (2 of 4 decision outcomes, 50% branch coverage) and the individual conditions are never tested.

Seeded logical error (mutant)	Result of the single test	Detected?
amount >= 1000 instead of > 1000	same (2000)	No
coupon || amount > 500 instead of &&	same (both true)	No
amount >= 500 instead of > 500	same	No
Decision 2 always true (condition removed)	same	No

2. Why statement coverage alone is insufficient

- It checks that a line *ran*, not that the condition that guards it is correct.
- An **if** without else has an implicit false branch with no statement, so it adds nothing to the statement count.
- Compound conditions can hide faulty sub-conditions (A || B is true whenever either is true).

3. Improved strategy: branch, path and condition tests

Decisions: 2, so V(G) = 3 independent paths and 2^2 = 4 complete paths.

TC	Amount	Member	Coupon	Decision 2, 3	Expected discount	Purpose
1	2000	F	T	T, T	15	Both true (original test)
2	100	F	F	F, F	0	Both false (branch coverage)
3	2000	F	F	T, F	10	Decision 2 only
4	600	F	T	F, T	5	Decision 3 only
5	100	F	T	F, F	0	Detects coupon || amount > 500
6	1000	F	F	F, F	0	Detects amount >= 1000
7	500	F	T	F, F	0	Detects amount >= 500
8	100	T	F	T, F	10	Member alone makes decision 2 true

Tests 1 to 4 give 100% branch coverage and all 4 paths; tests 5 to 8 add condition-level and boundary cases, which detect all 4 seeded errors.

4. Justification

Detection of seeded errors rises from 0 of 4 to 4 of 4; branch coverage rises from 50% to 100%; and every decision outcome and the boundaries 500 and 1000 are checked against expected discounts. **Interpretation:** statement coverage is the weakest white-box criterion; branch, condition and path coverage test the logic itself.

Question 5: Age Validation (0 to 100) with Only Two Classes

1. Errors in the partitioning strategy

No.	Error	Explanation
E1	"Positive numbers" mixes valid and invalid	It contains 1 to 100 (valid) and 101 and above (invalid)
E2	Zero belongs to neither class	0 is a valid age but is neither negative nor positive
E3	No upper-limit invalid class	101 and larger values are not recognised as invalid
E4	Other input types missing	Decimals (25.5), text (abc) and empty input are ignored
E5	No boundary value analysis	−1, 0, 1, 99, 100, 101 never tested

2. Why it fails valid vs invalid limits

The two classes are defined by the sign of the number, but the requirement is defined by the range 0 to 100. The class borders (0) and (100/101) lie inside a class instead of between classes, so a test using one value per class (for example −5 and 50) covers only 2 of 6 classes (33%) and none of the 6 boundaries.

3. Corrected partitions

Class	Range	Valid?	Representative
P1	< 0	No	−5
P2	0 to 100	Yes	50
P3	> 100	No	150
P4	Non-integer	No	25.5
P5	Non-numeric	No	abc
P6	Empty / null	No	(blank)

TC	Age	Type	Expected
1	−1	BVA min−	Rejected
2	0	BVA min	Accepted
3	1	BVA min+	Accepted
4	50	ECP P2	Accepted
5	99	BVA max−	Accepted
6	100	BVA max	Accepted
7	101	BVA max+	Rejected
8	150	ECP P3	Rejected
9	25.5	ECP P4	Rejected
10	abc	ECP P5	Rejected
11	(blank)	ECP P6	Rejected

4. Robustness justification

Seeded defect	Two-class design (−5, 50)	Corrected design
age < 100 (rejects 100)	Missed	Found by TC 6
age > 0 (rejects 0)	Missed	Found by TC 2

Seeded defect	Two-class design (−5, 50)	Corrected design
age <= 101 (accepts 101)	Missed	Found by TC 7
Detected	0 of 3	3 of 3

Interpretation: six partitions and six boundary values give complete, measurable coverage of the field, including inputs that are not numbers at all.

Question 6: Claim of Full Condition Coverage for if (A && B ...

*The question text is cut off after "if(A && B". This answer analyses the decision **if (A && B)**; the same method extends to longer expressions such as A && B || C.*

1. What condition coverage requires

Condition coverage: every atomic condition (A and B) must evaluate to both true and false at least once. It does **not** require the decision itself to take both outcomes.

2. Errors in the developer's claim

Test	A	B	A && B	Evaluated in Java/C-style short-circuit
T1	T	F	F	A is true, so B is evaluated: B = F
T2	F	T	F	A is false, so B is **not evaluated**

- **Error 1:** A is covered true (T1) and false (T2), but B is evaluated only in T1 (false). With short-circuit evaluation, **B = true is never actually evaluated**, so full condition coverage is not achieved.
- **Error 2:** even if the tests were counted as covering the conditions, the decision is false in both tests, so the true branch is never executed (branch coverage 50%).
- **Root cause:** condition coverage was confused with decision (branch) coverage, and short-circuit evaluation was ignored.

3. Corrected strategy

Test	A	B	Decision	Shows
T1	T	T	T	True branch; both conditions evaluated as true
T2	T	F	F	B alone makes the decision false
T3	F	— (not evaluated)	F	A alone makes the decision false

These 3 tests (n + 1 = 3 for 2 conditions) give condition coverage, decision (branch) coverage and **MC/DC**: A is shown to affect the decision independently by T1/T3 and B by T1/T2. A defect such as A || B instead of A && B is exposed by T2 (expected false, actual true). **Interpretation:** decision/condition coverage or MC/DC, not condition coverage alone, is needed for a compound condition.

Question 7: Decision Table: Premium Member and Coupon Applied

Assumed rules: Premium member 10% off; coupon 5% off; order above 2,000 gives 5% off; discounts add up but are capped at 15%. Conditions: C1 Premium, C2 Coupon, C3 Order > 2000.

1. Errors in the QA team's table

No.	Error	Effect
E1	Only single-condition rules listed (none, premium only, coupon only, bulk only)	Combined rules missing
E2	$2^3 = 8$ combinations not enumerated	Only 4 of 8 rules (50%) present
E3	The 15% cap rule is absent	Premium + coupon + bulk (20%) is never checked
E4	Expected prices not calculated per rule	Price errors cannot be detected

Missing rules: Premium + Coupon, Premium + Bulk, Coupon + Bulk and all three together; the price calculation errors appear exactly in those combinations.

2. Revised exhaustive decision table

Rule	Premium	Coupon	Order > 2000	Discount	Price used	Expected price
R1	N	N	N	0%	1000	1,000
R2	N	N	Y	5%	3000	2,850
R3	N	Y	N	5%	1000	950
R4	N	Y	Y	10%	3000	2,700
R5	Y	N	N	10%	1000	900
R6	Y	N	Y	15%	3000	2,550
R7	Y	Y	N	15%	1000	850
R8	Y	Y	Y	20% capped to 15%	3000	2,550

3. Effectiveness in preventing production defects

- Completeness: 8 of 8 rules (100%) against 4 of 8 (50%) before.
- R4, R6, R7 and R8 are the combination rules; R8 specifically tests the cap, where an uncapped calculation would give 3000 × 0.80 = 2,400 instead of 2,550 (a loss of 150 per order).
- Each rule has an exact expected price, so any calculation error shows as a failed test.

Interpretation: a decision table is only as good as its enumeration; listing all 2^n combinations turns hidden interactions into explicit tests.

Question 8: Loop Testing: Only n = 1 and n = max

Assumed loop: a loop that processes up to max = 10 items (for example computing an average); the tester ran n = 1 and n = 10.

1. Errors in the loop testing strategy

No.	Error	Typical defect missed
E1	n = 0 (loop skipped) not tested	Division by zero or null result when the list is empty
E2	n = max + 1 not tested	Array overflow or silent truncation
E3	n = 2 and typical n not tested	Defects that need a second iteration (state not reset)
E4	n = max − 1 not tested	Off-by-one in the exit condition
E5	No strategy for nested loops	Interaction between loops untested

2. Root causes

- The tester assumed that if the first and last iteration work, all iterations work.
- The three loop paths (skipped, entered, repeated) and the exit condition were not analysed as separate paths.
- Off-by-one conditions (< versus <=) were not considered, so values just outside the range were left out.

3. Corrected approach

Simple loop (max = 10): the set below tests the skip, once, twice, a typical number, one below the maximum, the maximum and one above.

TC	n	Purpose	Expected
1	0	Loop skipped	Empty result handled, no crash
2	1	One pass	Correct
3	2	Two passes (state reset)	Correct
4	5	Typical	Correct
5	9	max − 1	Correct
6	10	max	Correct
7	11	max + 1	Rejected or handled, no overflow

Nested loops (outer and inner, each max = 10), using the standard approach: (1) start with the innermost loop, set the outer loop to its minimum, and run the 7 simple-loop tests on the inner loop; (2) move outward, holding the inner loop at a typical value (for example 3) while running the 7 tests on the outer loop; (3) test both loops together at their minimum and both at their maximum. This needs 7 + 7 + 2 = 16 tests instead of the 7 × 7 = 49 combinations. **Concatenated loops** are tested independently if they do not share variables, and as nested loops if the second depends on the first.

4. Path integrity

Each test forces a distinct path through the loop: skip (decision false at once), enter and exit after one pass, repeated back edge, exit at the limit, and the rejection path beyond the limit. Together they exercise every edge of the loop's CFG, including the exit conditions, so the loop cannot behave differently on a path that was never run. **Interpretation:** the defects at n = 0 and n = max + 1 are found by the corrected 7-test set, which the original 2 tests could not find.

Question 9: Error Guessing for a Safety-Critical Braking System

1. Errors in relying on error guessing alone

No.	Error	Consequence
E1	Depends on the tester's memory and intuition	Different testers find different defects; results not reproducible
E2	No coverage measure	Impossible to show what was not tested
E3	No traceability to safety requirements	Cannot demonstrate that every requirement was verified
E4	Ignores state-dependent behaviour	Mode changes (ABS, assist, fault) are not systematically explored
E5	Ignores limits of sensor inputs	Threshold defects (slip, pressure, speed) missed
E6	Not acceptable as evidence for safety certification	Safety standards expect documented, structured test derivation

Impact on test adequacy: adequacy cannot be demonstrated, and the high-risk failure combinations (for example a sensor fault while ABS is active) depend on whether the tester happened to think of them.

2. Corrected strategy: BVA + State Transition Testing

Assumed thresholds: pedal pressure 0 to 100%; ABS engages when wheel slip exceeds 20%; maximum vehicle speed 200 km/h.

Input	BVA values	Expected behaviour
Pedal pressure %	−1, 0, 1, 99, 100, 101	−1 and 101: sensor error, fail-safe; 0: released; 1 to 100: braking
Wheel slip %	19, 20, 21	19 and 20: normal braking; 21: ABS active
Speed km/h	0, 1, 199, 200, 201	201: out-of-range reading handled safely

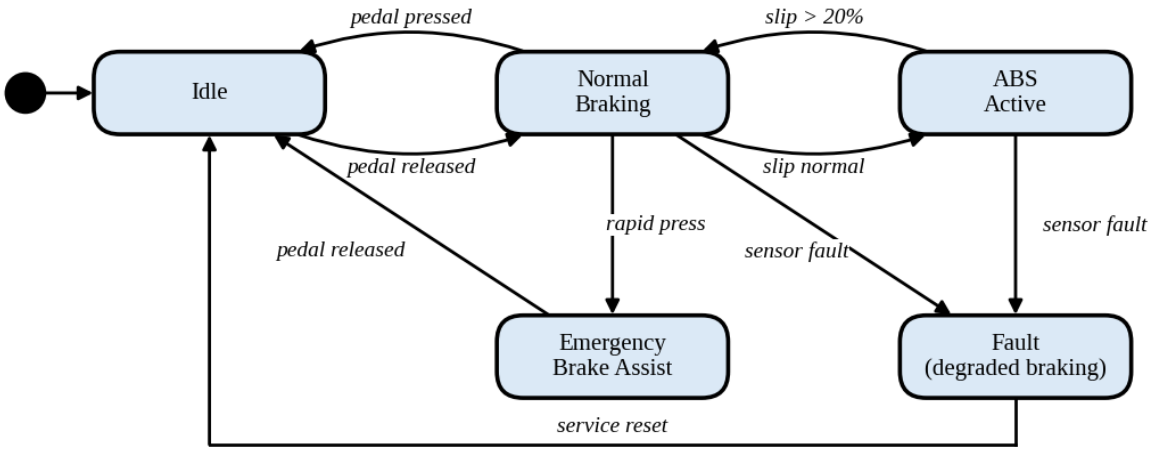


Figure 2: State-transition model of the braking controller

From	Event	To
Idle	pedal pressed	Normal Braking
Normal Braking	pedal released	Idle
Normal Braking	slip > 20%	ABS Active
ABS Active	slip normal	Normal Braking

From	Event	To
Normal Braking	rapid pedal press	Emergency Brake Assist
Emergency Brake Assist	pedal released	Idle
Normal Braking	sensor fault	Fault (degraded braking)
ABS Active	sensor fault	Fault (degraded braking)
Fault	service reset	Idle

Test type	Cases	Expected
All legal transitions (0-switch)	9 transitions	Correct next state
Illegal events	pedal released while Idle; slip > 20% while Idle; reset while Normal Braking	Ignored, state unchanged
Safety sequences (1-switch)	ABS Active then sensor fault; Normal then rapid press then sensor fault	Fault state with degraded braking still available
BVA at each threshold	14 values above	Correct state and response at each limit

3. Increased safety assurance

BVA proves every threshold, and the state model proves every mode change and the dangerous sequences; coverage can be reported (14 boundary values, 9 of 9 transitions, all illegal-event cells chosen), reviewed by safety assessors and repeated on every release. Error guessing is kept only as a supplement for unusual failure ideas. **Interpretation:** for safety-critical software the structured techniques provide evidence, while ad-hoc testing only provides opinion.

Question 10: CFG Missing the Else Branches of Nested if Statements

Assumed module (payment): `if (sessionValid) { if (amountValid) { if (balanceOK) process(); } } return;` The else parts are implicit (no code), so they were not drawn.

1. Errors in the graph construction

No.	Error	Effect
E1	Only the true edges of decisions 2, 3 and 4 were drawn	Chain 1→2→3→4→5→6 with 5 edges, V(G) = 5 − 6 + 2 = 1
E2	False (else) edges missing	No path models an invalid session, an invalid amount or an insufficient balance
E3	Implicit else treated as "nothing to draw"	An else with no statement still creates a branch

A single happy-path test executes every statement, so statement coverage is 100%, but only the 3 true outcomes of the 6 decision outcomes are taken: branch coverage is 50% overall and **0 of 3 (0%) on the error paths**.

2. Root causes

- Nodes were created only for statements; decisions without an else statement produce no node and therefore no edge.
- The rule "every decision has exactly two outgoing edges" was not applied.
- The graph was not checked with V(G) = decisions + 1 = 4.

3. Corrected CFG and independent paths

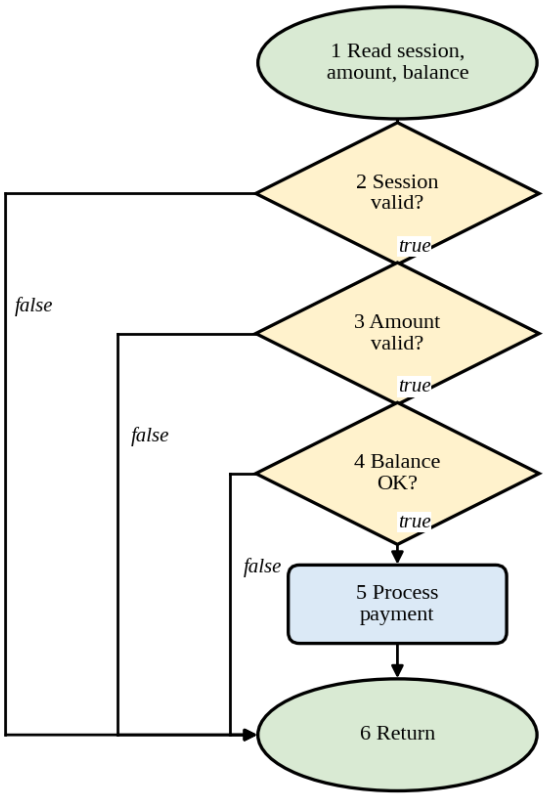


Figure 3: Corrected CFG with the false (else) edges of every decision

E = 8, N = 6, V(G) = 8 − 6 + 2 = **4**.

Path	Node sequence	Scenario	Test	Expected
P1	1-2-6	Session invalid	expired session	Rejected, no payment
P2	1-2-3-6	Amount invalid	amount = 0	Rejected, no payment
P3	1-2-3-4-6	Insufficient balance	amount > balance	Rejected, no payment
P4	1-2-3-4-5-6	All valid	valid request	Payment processed

4. Branch adequacy

Measure	Faulty graph and test	Corrected tests (P1 to P4)
Statement coverage	100%	100%
Branch coverage (6 outcomes)	50%	100%
Branch coverage on error paths (3 outcomes)	0%	100%
Independent paths	1 of 4	4 of 4

Interpretation: the corrected graph shows that the error paths exist and that all three need tests; the four tests give full branch adequacy. They also expose the code gap itself, since each rejection path currently falls through without any message.